

CHAPTER 3

TRANSPORT LAYER

Lecture-wise beginner-friendly notes
Lectures 09-13

How these notes are written: The uploaded lecture PDFs are the core source. Slide phrases are rewritten into simpler teaching language, visual diagrams are reconstructed as text, numerical examples are solved step by step, and formulas are grouped at the end of each lecture. Chapter 4 content in Lecture 13 is deliberately excluded from the Chapter 3 notes.

Designed to be understandable even if this is your first time studying transport-layer networking.

Study map

Lecture	Main topics
Lecture 09 - 15 Sep 2026	Transport-layer actions; TCP vs UDP; multiplexing/demultiplexing; ports; UDP; checksum; reliable-data-transfer idea; FSM basics
Lecture 10 - 17 Sep 2026	rdt1.0 -> rdt3.0; ACK/NAK; sequence numbers; timeout; loss scenarios; stop-and-wait performance; pipelining
Lecture 11 - 24 Sep 2026	Go-Back-N; Selective Repeat; SR sequence-number dilemma; comparison; TCP overview; TCP header; sequence and ACK numbers
Lecture 12 - 29 Sep 2026	TCP RTT/timeout; retransmission; ACK generation; flow control; connection management; congestion control; Tahoe vs Reno
Lecture 13 - 01 Oct 2026	TCP Reno numerical; evolving transport; QUIC; no HOL blocking; QUIC numerical; TCP throughput; long-fat-pipe example

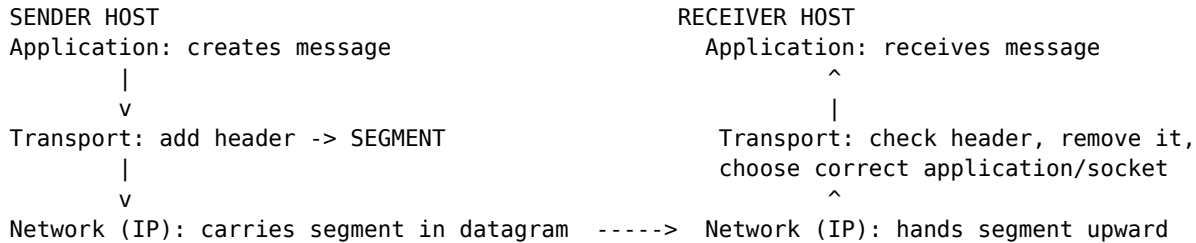
Recommended study order: Read the explanation first, then cover the worked example yourself, and finally use the formula sheet + quick self-check. The text diagrams are intentionally simple: focus on direction, state, and what each arrow means rather than memorizing artwork.

Lecture 09 - Transport Layer Foundations, Multiplexing, UDP, and RDT Introduction

Source lecture: 15 September 2026. Chapter roadmap focus: transport-layer services -> multiplexing/demultiplexing -> UDP -> start of reliable data transfer.

1. What the transport layer actually does

The transport layer sits between applications and the network layer. Applications create messages; the transport layer turns those messages into transport-layer segments and gives them to IP. At the destination, the reverse happens.



- Sender side: receives an application-layer message, chooses transport-header field values, creates a segment, then passes the segment to the network layer (IP).
- Receiver side: receives a segment from IP, checks the transport header, extracts the application data, and delivers that data to the correct application through a socket.

Beginner translation: A segment is simply application data plus a transport-layer header. The header contains control information such as port numbers, sequence numbers, checksums, etc., depending on the protocol.

2. The two main Internet transport protocols: TCP and UDP

Feature	TCP	UDP
Delivery	Reliable and in order	Best effort: data may be lost or arrive out of order
Connection setup	Yes	No
Flow control	Yes - protects the receiver	No built-in flow control
Congestion control	Yes - reacts to network congestion	No built-in congestion control
Style	More features/state	"No-frills" / minimal transport service

Neither TCP nor UDP gives an application a guaranteed end-to-end delay or a guaranteed bandwidth. A protocol may try to perform well, but the Internet transport service does not promise a fixed delay or fixed data rate.

3. Multiplexing and demultiplexing

A host can run many applications at the same time: a browser, video app, game, messaging app, etc. The transport layer therefore needs a way to combine outgoing data from many applications and later separate incoming data back to the correct application.

```

SENDER (multiplexing)                                RECEIVER (demultiplexing)
App A --\                                           /--> App A
App B ----> [ Transport layer ] ---- network ----> [ Transport layer ] --> App B
App C --/                                           \--> App C

```

Multiplexing = many application sockets -> outgoing segments

Demultiplexing = incoming segment -> the correct socket/application

Feature	Multiplexing	Demultiplexing
Direction	Outgoing - sender	Incoming - receiver
Main action	Gather data from many applications/sockets and add transport headers	Read header values and distribute each segment to the correct socket/application

Analogy used by the lecture visuals: Multiplexing is like several roads joining one highway. Demultiplexing is like highway exits directing traffic to different destinations. The useful networking idea is not the picture itself - it is the many-to-one vs one-to-many direction.

4. Ports: how the transport layer identifies a process

An IP address identifies a host/interface on the network. A port number helps identify which process or service inside that host should get the transport-layer data.

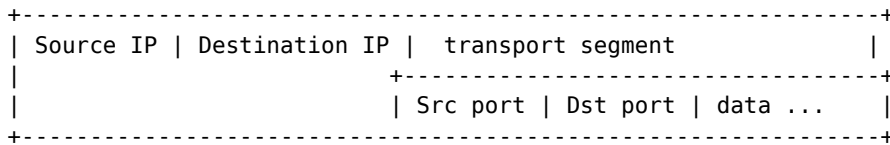
- Port numbers are 16-bit integers: 0 through 65,535.
- Client programs usually choose a temporary (ephemeral) source port.
- Server services normally listen on known/pre-assigned ports so clients know where to contact them.

Range	Port numbers	Meaning / lecture examples
Well-known	0-1023	Assigned/controlled by IANA. Examples in lecture: HTTP 80, FTP 21.
Registered	1024-49,151	Registered to avoid duplication; lecture notes describe these as not assigned/controlled like well-known ports.
Dynamic / Private	49,152-65,535	Not controlled/registered for a fixed service; commonly used as ephemeral ports.

5. How demultiplexing works

An IP datagram contains source and destination IP addresses and carries a transport segment. The transport segment contains source and destination port numbers. The receiving host uses these values to decide which socket gets the data.

IP DATAGRAM



5.1 UDP / connectionless demultiplexing

For UDP, the receiver mainly directs a segment according to the local destination port. The lecture describes a UDP socket using a 2-tuple: destination IP address + destination port number.

- When sending a UDP datagram, the sender specifies the destination IP address and destination port.
- At the receiver, UDP checks the destination port and delivers the segment to the socket bound to that port.
- Different clients can send from different source IPs and source ports to the same destination port; those datagrams can still arrive at the same UDP socket.

Gaming-server example: If many players send UDP packets to one game-server UDP socket, the transport layer can deliver all of them to that socket. If the application needs to keep separate per-player state, the application itself must distinguish the clients (for example from address information carried with each datagram).

5.2 TCP / connection-oriented demultiplexing

A TCP connection is identified using four values, so a server can keep many simultaneous client connections separate even when they all connect to the same server port.

TCP CONNECTION IDENTIFIER (4-tuple)

(source IP, source port, destination IP, destination port)

Client A: (A, 9157, B, 80) ---- Client C: (C, 5775, B, 80) -----> Server B, port 80, but
DIFFERENT TCP sockets

Client C: (C, 9157, B, 80) ----/

- UDP demultiplexing in these slides: destination port identifies the receiving socket (with local destination IP as part of the socket context).
- TCP demultiplexing: uses the full 4-tuple, so different clients can share the same server destination port without being mixed together.

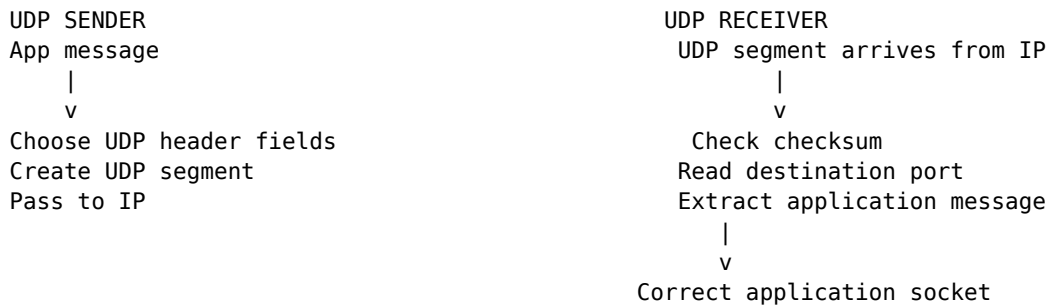
6. UDP: User Datagram Protocol

UDP is intentionally simple. It adds only a small amount of transport functionality on top of IP. There is no connection-establishment handshake and no per-connection state like TCP maintains.

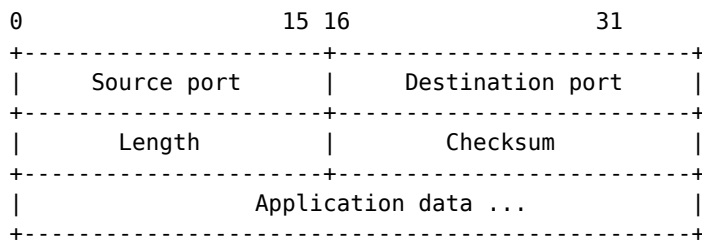
- Best-effort delivery: UDP segments can be lost.
- No in-order guarantee: an application may receive segments in a different order than they were sent.

- No connection establishment: this avoids setup RTT delay.
- Small fixed header and simple sender/receiver behavior.
- No built-in congestion control; a UDP application can send as fast as it chooses unless the application implements its own control.
- Lecture examples of UDP use: streaming multimedia, DNS, SNMP, and HTTP/3 (through QUIC).
- If an application wants reliability or congestion control while using UDP, it can implement those functions at the application layer.

6.1 UDP sender and receiver actions



6.2 UDP header



UDP header size = 8 bytes (four 16-bit fields)

- Length field = total UDP segment length in bytes, including the 8-byte header.
- Therefore, UDP application-data length = UDP Length - 8 bytes.
- Checksum is used to detect bit corruption.

7. Internet checksum

The checksum is an error-detection value. The sender treats the protected contents as 16-bit words, adds them using one's-complement addition, and places the one's complement of the sum in the checksum field. The receiver performs the corresponding calculation to see whether corruption is detected.

1. Split the protected contents into 16-bit integers.
2. Add the 16-bit words. If there is a carry out of the most-significant bit, wrap that carry around and add it back into the low 16 bits.
3. Take the one's complement (flip every 0 to 1 and every 1 to 0). That becomes the checksum.
4. Receiver recomputes the checksum. If the result does not match the received checksum field, an error is detected.

Important limitation: A matching checksum does not prove that the data is definitely correct. The lecture explicitly shows that some combinations of bit flips can leave the checksum unchanged, so

checksum protection is useful but not perfect.

Worked example: adding two 16-bit words

```
Word 1:      1110 0110 0110 0110
Word 2:      1101 0101 0101 0101
-----
Raw sum:    1 1011 1011 1011 1011  <- 17th-bit carry = 1
```

```
Wrap the carry around:
Low 16 bits: 1011 1011 1011 1011
Carry:      0000 0000 0000 0001
-----
Wrapped sum: 1011 1011 1011 1100
```

```
One's complement (flip bits):
Checksum:   0100 0100 0100 0011
```

The key step is the wraparound carry. Ordinary binary addition would keep the 17th bit, but one's-complement addition folds that carry back into the low-order end before complementing the result.

8. Why reliable data transfer is a separate problem

Applications often want the illusion of a reliable channel: data sent by one side should arrive correctly and in the proper logical sequence. The underlying network, however, can be unreliable. A reliable-data-transfer (rdt) protocol is the logic placed at the sender and receiver to create a reliable service on top of an unreliable channel.

What the application WANTS to see:

```
Sender application  ===== reliable channel =====> Receiver application
```

What the protocol ACTUALLY has underneath:

```
Sender app -> sender rdt -> [unreliable network channel] -> receiver rdt -> receiver app
```

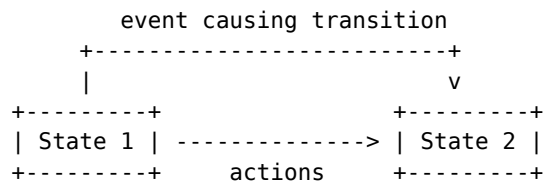
- The difficulty of the rdt protocol depends on what the underlying channel can do wrong: corrupt bits, lose packets, reorder packets, etc.
- Sender and receiver do not magically know each other's state. If the sender needs to know whether a packet arrived, that fact must be communicated through protocol messages such as ACKs.

9. rdt interfaces used in the slides

Interface	Who calls it / what it means
rdt_send(data)	Called from above at the sender (for example by the application) to give data to the reliable transport logic.
udt_send(packet)	Called by rdt to send a packet through the underlying unreliable channel.
rdt_rcv(packet)	Triggered when a packet arrives from the underlying channel at the receiver.
deliver_data(data)	Called by receiver-side rdt to give correctly received data to the upper layer/application.

10. Finite State Machines (FSMs)

The lectures use FSMs to describe protocol behavior. An FSM is an abstract machine that is in exactly one of a finite number of states at a time. An event can cause a transition to another state, and actions can happen during that transition.



When you read an rdt FSM, always ask:

- 1) What state am I in?
 - 2) What event happened?
 - 3) What action is performed?
 - 4) What state comes next?
- FSM definition in the lecture: list of states, an initial state, the input/event that triggers each transition, and the actions produced by that event.
 - Everyday examples from the slide: vending machine, elevator, traffic signal, turnstile.

Lecture 09 - Formula and rule sheet

Formula / rule	Meaning / when to use
Port range	16-bit port number => 0 through 65,535.
UDP data length	Application data bytes = UDP Length field - 8 bytes.
UDP header size	8 bytes total.
Internet checksum	One's-complement sum of 16-bit words (with end-around carry), then complement the final 16-bit sum.
UDP demultiplexing key	Lecture model: destination socket identified by destination IP + destination port (2-tuple).
TCP demultiplexing key	(source IP, source port, destination IP, destination port) - 4-tuple.

Quick self-check

- Why can two different TCP clients connect to the same server port 80 without their data being mixed?
- What is the difference between multiplexing and demultiplexing?
- Why can a matching Internet checksum still fail to prove that data is error-free?
- What is the purpose of `rdt_send()`, `udt_send()`, `rdt_rcv()`, and `deliver_data()`?

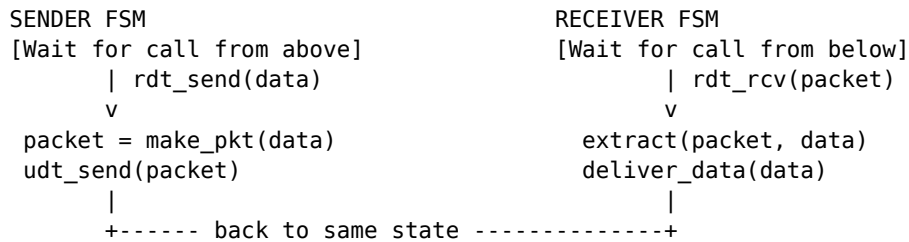
Administrative/non-study slides: The final slides contain Assignment #2 and Quiz #2 announcements for Chapter 2; they are not Chapter 3 technical content.

Lecture 10 - Reliable Data Transfer: rdt1.0 to rdt3.0 and Pipelining

Source lecture: 17 September 2026. Focus: progressively adding mechanisms as the channel becomes less reliable.

1. rdt1.0 - perfectly reliable channel

rdt1.0 is the easiest starting point because the underlying channel is assumed to have no bit errors and no packet loss. Since nothing can go wrong, the sender simply packs the data and sends it, and the receiver extracts the data and delivers it upward.



Why begin with an unrealistic case?: Because each later rdt version adds exactly the new machinery needed for a new type of failure. This makes it easier to understand why checksums, ACKs, sequence numbers, and timers exist.

2. rdt2.0 - channel can corrupt bits

Now packets can arrive with flipped bits. A checksum can detect corruption, but detection alone is not enough: the sender must learn whether the receiver got a good or bad packet.

- ACK (acknowledgement): receiver says the packet was received correctly.
- NAK (negative acknowledgement): receiver says the packet was corrupted.
- Retransmission: on NAK, the sender sends the same packet again.
- Stop-and-wait behavior: the sender sends one packet, then waits for the receiver's ACK/NAK before moving on.

New mechanism in rdt2.0	Purpose
Error detection	Checksum reveals that a packet was corrupted.
Receiver feedback	ACK/NAK communicates receiver state back to sender.
Retransmission	Sender resends a packet that arrived in error.

3. The fatal flaw in rdt2.0: what if the ACK/NAK is corrupted?

If the receiver's feedback itself is corrupted, the sender no longer knows whether the data packet was accepted. Blindly retransmitting can create a duplicate at the receiver. This creates the need for sequence numbers.

```

Sender sends packet P
      |
      v
Receiver gets P correctly -> sends ACK
      |
      X ACK becomes corrupted
      |
Sender cannot tell whether receiver accepted P.
If sender resends P, receiver must recognize: "this is the SAME packet again."

```

4. rdt2.1 - sequence numbers handle duplicates

rdt2.1 adds a sequence number to each packet. Because this is stop-and-wait, only two sequence numbers, 0 and 1, are needed. The sender alternates 0,1,0,1,... and the receiver remembers which one it expects next.

- If ACK/NAK is corrupted, the sender retransmits the current packet.
- If the receiver sees the same sequence number again when it was expecting the other one, it knows the arriving packet is a duplicate and does not deliver the data to the application again.
- Receiver still sends feedback so the sender can continue.

Why only 0 and 1?: At most one unacknowledged data packet exists in stop-and-wait. The receiver only needs to distinguish "new packet" from "retransmission of the previous packet," so alternating one bit is sufficient.

5. rdt2.2 - remove NAKs and use ACKs only

rdt2.2 provides the same basic function as rdt2.1 without explicit NAK messages. **If a packet is bad or unexpected, the receiver repeats an ACK for the last packet it received correctly.** The ACK contains the sequence number being acknowledged.

- A duplicate/old ACK at the sender plays the role a NAK used to play: it tells the sender the current packet was not successfully advanced, so the sender retransmits it.
- The lecture notes that TCP follows this ACK-only style rather than using NAK packets.

6. rdt3.0 - packets and ACKs can be lost

Checksums, sequence numbers, ACKs, and retransmission solve corruption and duplication, but loss creates a new problem: the sender might wait forever for an ACK that will never arrive. rdt3.0 therefore adds a timer.

- After sending a packet, start a countdown timer.
- If the correct ACK arrives in time, stop the timer and move to the next sequence number.
- If the timer expires first, assume the data packet or its ACK may have been lost and retransmit the packet.
- If the original packet/ACK was only delayed rather than lost, the retransmission can create a duplicate - sequence numbers let the receiver safely detect and discard the duplicate data.

7. rdt3.0 scenarios you should be able to trace

Scenario	What happens
No loss	pkt0 -> ACK0 -> pkt1 -> ACK1. Sender progresses

	normally.
Data packet lost	Sender waits, timer expires, sender retransmits the missing packet, then receives ACK.
ACK lost	Receiver may already have accepted the packet. Sender times out and retransmits; receiver recognizes the duplicate and sends the ACK again without redelivering duplicate data.
Premature timeout / delayed ACK	Sender retransmits even though the original was not truly lost. Duplicate detection makes this safe; an old ACK that arrives later can be ignored if it no longer matches the sender state.

Example: ACK loss

```

Sender                                     Receiver
pkt1 -----> receive pkt1, deliver once
      <----- ACK1 ----- X          ACK is lost
[timer expires]
pkt1 -----> sees duplicate seq=1, DO NOT deliver again
      <----- ACK1 ----- sender can continue

```

8. Why stop-and-wait wastes a fast link

The lecture uses a 1 Gbps link, 15 ms one-way propagation delay, and an 8000-bit packet. The sender needs only a few microseconds to push the packet bits onto the link, then sits idle for almost a full RTT waiting for an ACK.

Worked numerical: stop-and-wait utilization

1. Transmission delay: $D_{trans} = L/R = 8000 \text{ bits} / 10^9 \text{ bits/s} = 8 \times 10^{-6} \text{ s} = 8 \text{ microseconds} = 0.008 \text{ ms}$.
2. One-way propagation delay is 15 ms, so RTT is approximately $2 \times 15 \text{ ms} = 30 \text{ ms}$ (the lecture ignores ACK transmission time here).
3. One stop-and-wait cycle lasts $RTT + L/R = 30 \text{ ms} + 0.008 \text{ ms} = 30.008 \text{ ms}$.
4. Sender is actively transmitting only for $L/R = 0.008 \text{ ms}$.
5. Utilization = $0.008 / 30.008 = \text{about } 0.00027 = \text{about } 0.027\%$.

Interpretation: A 1 Gbps link is available, yet the sender is busy transmitting only about 0.027% of the cycle in this example. The protocol, not the physical link speed, becomes the bottleneck.

9. Pipelining: keep several packets in flight

Pipelining removes the one-packet-at-a-time restriction. The sender is allowed to transmit multiple packets before their ACKs return. This fills the long waiting period with useful transmissions.

- More sequence numbers are required because several packets can be outstanding at once.
- The sender and/or receiver need buffers to keep track of multiple in-flight packets.

STOP-AND-WAIT:

Send P0
wait...
wait...
ACK0 arrives
Send P1

PIPELINING:

Send P0
Send P1
Send P2
... ACKs return later
Link is used much more continuously

Worked numerical: 3-packet pipeline

1. From the same example, one packet transmission time $L/R = 0.008$ ms.
2. Three packets occupy $3L/R = 3 \times 0.008 = 0.024$ ms of sending time per RTT-sized cycle.
3. $U_{\text{sender}} = (3L/R) / (RTT + L/R) = 0.024 / 30.008 = \text{about } 0.00080$, or about 0.080%.
4. This is approximately 3 times the stop-and-wait utilization, exactly the qualitative point of the slide.

Slide arithmetic note: The pipelining slide visually prints a numerator as .0024 while also stating a result near 0.00081. Using the lecture's own $L/R = 0.008$ ms, the consistent numerator is 0.024 ms, giving about 0.00080. The intended conclusion - roughly a factor-of-3 improvement - is unchanged.

Lecture 10 - Formula and rule sheet

Formula / rule	Meaning / when to use
Transmission delay	$D_{\text{trans}} = L / R$
Round-trip time in this example	$RTT = 2 \times \text{one-way propagation delay}$ (ACK transmission time ignored in the slide example).
Stop-and-wait utilization	$U_{\text{sender}} = (L/R) / (RTT + L/R)$
N-packet pipeline utilization (same simplified model)	U_{sender} approximately $= N(L/R) / (RTT + L/R)$, until utilization reaches the physical limit of 1.
Sequence numbers in stop-and-wait rdt2.1/2.2/3.0	Alternate 0 and 1.
Timeout rule	If the correct ACK is not received before timeout, retransmit the outstanding packet.

Quick self-check

- Why is a checksum alone not enough for reliable transfer?
- Why does rdt2.1 need sequence numbers?
- How does rdt2.2 replace NAK with ACK behavior?
- In rdt3.0, how can the receiver safely handle a retransmission caused by a delayed ACK rather than a real loss?
- Why does pipelining increase utilization on a large-RTT link?

Administrative/non-study slides: The final slides are Midterm I / Quiz 2 announcements, not Chapter 3 technical material.

1. Two generic pipelined protocols

2. Go-Back-N (GBN) / sliding-window protocol

- If $N = 1$, GBN reduces to stop-and-wait. Pipelining therefore needs $N > 1$.
- With a k -bit sequence-number field, sequence numbers range from 0 to $2^k - 1$ and eventually wrap around.
- ACK(n) is cumulative: it means every packet through sequence number n has been received in order.
- GBN normally keeps one timer for the oldest outstanding packet.
- If packet n times out, retransmit n and every later outstanding packet currently in the sender window.

0		base-1		base	nextseqnum-1		nextseqnum	base+N-1		base+N
		ACKed		sent, not	ACKed		allowed to	send now		not allowed yet
				(in flight)						

2.2 GBN receiver behavior

- Correct next in-order packet: accept/deliver it and ACK the new highest in-order sequence number.
- Out-of-order packet: the lecture says an implementation may discard or buffer, but the standard GBN behavior shown in the example discards it and re-sends the ACK for the highest in-order packet.

- Duplicate ACKs can therefore occur repeatedly while a gap is missing.

2.3 GBN example: packet 2 is lost, N = 4

```

Sender                                     Receiver
pkt0 -----> receive 0, ACK0
pkt1 -----> receive 1, ACK1
pkt2 ----- X (lost)
pkt3 -----> out of order -> discard, re-ACK1
ACK0/ACK1 slide sender window forward enough to send pkt4,pkt5
pkt4 -----> out of order -> discard, re-ACK1
pkt5 -----> out of order -> discard, re-ACK1

```

Timer for oldest missing packet expires (pkt2)
 Sender retransmits pkt2, pkt3, pkt4, pkt5
 Receiver now accepts them in order and ACKs progress.

Core GBN idea: One loss can cause several correctly transmitted later packets to be sent again. That is the price of simple cumulative ACK and simple receiver logic.

3. Selective Repeat (SR)

Selective Repeat tries to avoid unnecessary retransmissions. The receiver individually acknowledges correctly received packets, even when they arrive out of order, and buffers them until missing earlier data arrives.

- Sender conceptually maintains a separate timer for each unacknowledged packet.
- Timeout for packet n -> retransmit only packet n, not the whole remaining window.
- Receiver ACKs each correctly received packet individually.
- Receiver buffers out-of-order packets so it can later deliver a continuous in-order block to the application.
- Sender and receiver window sizes are both N in the lecture model.

3.1 SR sender and receiver rules

Side / event	Action
Sender: data from above	If the next sequence number is inside the current send window, build and send the packet.
Sender: timeout(n)	Retransmit packet n and restart n's timer.
Sender: ACK(n) inside window	Mark n received. If n was the smallest unACKed packet, move sendbase forward to the next unACKed sequence number.
Receiver: n in current receive window	Send ACK(n). If out of order, buffer it; if it fills the next required position, deliver it and any newly contiguous buffered packets.
Receiver: n in previous receive window	ACK it again - it may be a retransmission whose earlier ACK was lost.
Receiver: all other sequence numbers	Ignore.

3.2 SR example: packet 2 is lost, N = 4

```

Sender                                     Receiver
pkt0 -----> receive, ACK0
pkt1 -----> receive, ACK1
pkt2 ----- X
pkt3 -----> receive OUT OF ORDER, buffer, ACK3
pkt4 -----> buffer, ACK4
pkt5 -----> buffer, ACK5

Only pkt2 times out.
pkt2 -----> receive pkt2; now 2,3,4,5 are contiguous
                                     deliver 2,3,4,5; ACK2

```

Sender does NOT retransmit 3,4,5 because their individual ACKs were recorded.

4. The Selective Repeat sequence-number dilemma

Because sequence numbers eventually wrap around, an old retransmission can have the same sequence number as a genuinely new packet. If the receive window is too large, the receiver may not be able to tell the two apart.

- Lecture example: sequence numbers 0,1,2,3 (sequence-number space size 4), but window size is 3.
- After wraparound, sequence number 0 becomes valid again while an old packet 0 might still reappear due to retransmission.
- The receiver cannot see the sender's internal history, so the two cases can look identical.

Required relationship: For Selective Repeat, window size must be less than or equal to half the sequence-number space. In symbols: $N \leq S/2$, where S is the number of distinct sequence numbers. Equivalently, the sequence-number space must satisfy $S \geq 2N$.

- This is why the comparison slide lists the minimum sequence-number space for SR as $2N$.

5. Stop-and-Wait vs GBN vs SR

Property	Stop-and-Wait	Go-Back-N	Selective Repeat
Sender window size	1	N	N
Receiver window size	1	1	N
Minimum sequence-number space	2	$N+1$	$2N$
Efficiency shown in lecture	$1/(1+2a)$	$N/(1+2a)$	$N/(1+2a)$
ACK style	Individual	Cumulative	Individual
Out-of-order receive support	No useful window	In-order delivery only	Can receive/buffer out of order
Retransmissions after one packet drop	1	Up to N in the illustrated comparison	1 lost packet

Here a = propagation delay / transmission delay. These efficiency expressions are the simplified formulas shown in the lecture comparison table.

6. TCP overview

TCP is a connection-oriented, reliable byte-stream transport protocol. It combines ideas from reliable transfer, pipelining, flow control, and congestion control.

- Point-to-point: one sender endpoint and one receiver endpoint per TCP connection.
- Reliable, in-order byte stream: TCP numbers bytes, not application messages; the byte stream does not preserve message boundaries for the application.
- Full duplex: both endpoints can send data in the same connection.
- Cumulative ACKs are used.
- Pipelining is used; flow-control and congestion-control windows affect how much data can be in flight.
- Connection-oriented: a 3-way handshake initializes state before normal data exchange.
- Flow controlled: sender is prevented from overwhelming the receiver.
- MSS (Maximum Segment Size): maximum amount of application data carried in a TCP segment; it is based on path MTU and does not mean the total segment size including the TCP header.

7. TCP segment header

Source port		Destination port	
Sequence number			
Acknowledgement number			
Hdr	flags	Receive window (rwnd)	
len			
Checksum		Urgent data pointer	
Options (0 to 40 bytes) ...			
Application data ...			

- Sequence number: byte-stream number of the first data byte in this segment.
- Acknowledgement number: sequence number of the next byte expected from the other side; ACKs are cumulative.
- rwnd: receiver-advertised window used for flow control.
- Checksum: Internet checksum.
- RST, SYN, FIN: connection-management flags.
- ACK flag: says the acknowledgement field is valid; PSH and URG have the meanings discussed below; C/E are used for congestion notification in the diagram.

7.1 TCP header length

- Header-length field is 4 bits.
- Minimum TCP header = 20 bytes; maximum = 60 bytes.
- The first 20 bytes are always present; options can add 0 through 40 bytes.
- Scaling factor is 4 bytes: TCP header length in bytes = header-length field value x 4.

Example: If the header-length field contains decimal 5, the TCP header is $5 \times 4 = 20$ bytes. If it

contains 15, the header is $15 \times 4 = 60$ bytes.

7.2 PSH vs URG flags - as presented in the lecture

PSH	URG
Pushes buffered data onward to the network layer at the sender / application layer at the receiver.	Marks urgent data so it can be given to the application immediately; the urgent-data pointer identifies the urgent position.
Lecture slide describes PSH data as delivered in sequence.	Lecture slide describes urgent data as able to be delivered out of sequence.

8. TCP sequence and acknowledgement numbers

TCP numbers bytes in the byte stream. A segment's sequence number is the number of its first data byte. The ACK number tells the other side the next byte expected.

Sender byte stream:

... [already ACKed] [sent, not ACKed] [usable but not yet sent] [outside window] ...
 <----- current send window ----->

If ACK = 100, receiver is saying: "I have everything through byte 99; please send byte 100 next."

- TCP ACKs are cumulative: an ACK for byte y implicitly acknowledges all earlier bytes in the connection that form the contiguous prefix.
- The TCP specification leaves exact handling of out-of-order segments to the implementation; the lecture lists two basic choices: discard or buffer.

Worked example: simple Telnet-style character echo

```

Host A                               Host B
User types 'C'
Seq=42, ACK=79, data='C' ----->
                                <----- Seq=79, ACK=43, data='C'
Seq=43, ACK=80 ----->
```

Reasoning:

- A sent one byte beginning at seq 42, so B now expects 43.
- B echoes one byte beginning at seq 79, so A now expects 80.
- ACK numbers therefore move by one byte.

Lecture 11 - Formula and rule sheet

Formula / rule	Meaning / when to use
k-bit sequence numbers	Range = 0 through $2^k - 1$; sequence-number space size $S = 2^k$.
GBN cumulative ACK	ACK(n) acknowledges all packets through n.
SR safe window rule	$N \leq S/2$, or equivalently $S \geq 2N$.
Minimum sequence-number space	Stop-and-Wait: 2; GBN: $N+1$; SR: $2N$ (as shown in lecture comparison).

a	$a = \text{propagation delay} / \text{transmission delay}.$
Simplified stop-and-wait efficiency	$1 / (1 + 2a).$
Simplified pipelined efficiency shown	$N / (1 + 2a)$ for GBN and SR in the comparison table.
TCP header length	Header bytes = header-length field $\times$ 4; valid lecture range 20-60 bytes.

Quick self-check

- Why does GBN retransmit more data than SR after one packet loss?
- What does a cumulative ACK mean?
- Why must SR use a sequence-number space at least twice the window size?
- If a TCP segment starts at sequence number 500 and carries 100 bytes, what sequence number would the receiver expect next?
- What is the difference between TCP MSS and total TCP segment size?

Lecture 12 - TCP Reliability, Flow Control, Connection Management, and Congestion Control

Source lecture: 29 September 2026. This lecture ties together TCP timing, ACK behavior, receiver protection, handshaking, and congestion response.

1. TCP RTT estimation and timeout

TCP needs a retransmission timeout that is long enough to avoid constant false retransmissions but short enough to react reasonably quickly to real loss. Because RTT varies over time, TCP estimates it rather than using one fixed measurement.

- **SampleRTT**: measured time from sending a segment until its ACK is received. The slide says to ignore retransmitted segments when taking this measurement.
- **EstimatedRTT**: a smoothed value made from the previous estimate plus the newest sample.
- **DevRTT**: a smoothed estimate of how much the samples vary around the estimated RTT; more variation means a larger safety margin is needed.

$$\text{EstimatedRTT}_{\text{new}} = (1 - \alpha) * \text{EstimatedRTT}_{\text{old}} + \alpha * \text{SampleRTT}_{\text{current}}$$

Typical $\alpha = 0.125$

$$\text{DevRTT}_{\text{new}} = (1 - \beta) * \text{DevRTT}_{\text{old}} + \beta * |\text{SampleRTT}_{\text{current}} - \text{EstimatedRTT}_{\text{old}}|$$

Typical $\beta = 0.25$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

Meaning: EstimatedRTT follows the general RTT trend. DevRTT tells TCP how unstable that trend is. The timeout is the estimate plus a safety margin so normal RTT variation is less likely to trigger a false timeout.

2. Doubling the timeout after repeated timeouts

When a retransmission timeout occurs, the lecture states that TCP doubles the next timeout interval rather than immediately recomputing it from EstimatedRTT and DevRTT. Repeated timeouts therefore create exponentially larger waiting intervals. This is a limited form of congestion control because the sender backs off instead of retransmitting aggressively.

- After normal events such as new data/ACK progress, the timer again uses the current EstimatedRTT and DevRTT-based TimeoutInterval.

3. Simplified TCP sender behavior

Event	Sender action
Application gives TCP new data	Create a segment; sequence number is the byte number of the first data byte. Start the timer if it is not already running. The single timer can be thought of as timing the oldest unACKed segment.
Timeout	Retransmit the segment whose loss is being timed and restart the timer.

Cumulative ACK arrives	If the ACK newly acknowledges previously unACKed bytes, advance the known ACKed point. If unACKed data remains, run/restart the timer; otherwise stop it.
------------------------	---

4. TCP receiver ACK generation (RFC 5681 table from the slide)

Receiver event	Receiver action shown in lecture
In-order segment arrives with expected sequence number; all earlier data already ACKed	Delayed ACK: wait up to 500 ms for the next segment. If no next segment arrives, send ACK.
In-order segment arrives and one other in-order segment already has an ACK pending	Immediately send one cumulative ACK covering both in-order segments.
Out-of-order segment with sequence number higher than expected arrives; a gap is detected	Immediately send a duplicate ACK naming the next byte still expected.
Segment arrives that partially or completely fills a gap	Immediately send ACK, provided the arriving segment begins at the lower end of the gap.

5. TCP retransmission scenarios

5.1 Lost ACK

Suppose Host A sends Seq=92 carrying 8 bytes, so the correct next expected byte is 100. If ACK=100 is lost, A eventually times out and retransmits the segment. B recognizes the data and sends ACK=100 again. The duplicate data is not supposed to be delivered twice to the application.

5.2 Premature timeout and cumulative ACK

If A sends Seq=92 (8 bytes) and then Seq=100 (20 bytes), an early timeout may make A retransmit the first segment even though B has already progressed. B can answer with the newest cumulative ACK (for example ACK=120), which tells A that all bytes before 120 are already covered.

5.3 A later cumulative ACK can cover an earlier lost ACK

If ACK=100 is lost but the receiver later sends ACK=120 after receiving the next in-order data, ACK=120 also confirms all earlier contiguous bytes. The sender therefore does not need a separate ACK=100 to make progress.

6. TCP fast retransmit

Waiting for a timeout can be slow. Duplicate ACKs are useful evidence that later segments are reaching the receiver while one earlier segment is missing.

- If the sender receives three additional ACKs for the same byte position (triple duplicate ACK), it treats the smallest-sequence unACKed segment as likely lost.
- TCP retransmits that missing segment immediately instead of waiting for the timeout.

Why duplicate ACKs are evidence: The receiver is repeatedly saying "I am still waiting for the same byte," while later packets are arriving. After three duplicates, TCP assumes a gap is very likely.

7. Is TCP GBN or Selective Repeat?

The lecture categorizes TCP loss/error recovery as a hybrid of GBN and SR rather than a pure implementation of either one. TCP uses cumulative ACKs (GBN-like) but modern TCP can avoid blindly retransmitting the entire remaining window and can buffer/recover more selectively depending on implementation and extensions.

8. Flow control vs congestion control

Flow control	Congestion control
Protects one receiver from one sender that is delivering data too quickly.	Protects the network from too many senders putting too much traffic into the network.
Feedback meaning: receiver buffer is filling/overflowing.	Feedback/evidence meaning: network is congested; loss/delay or explicit signals indicate trouble.

9. TCP flow control with rwnd

Incoming TCP data is placed in a receive buffer before the receiving application reads it. If the network delivers data faster than the application consumes it, the buffer can fill. TCP prevents overflow by letting the receiver advertise how much free buffer space remains.

```

Network -> [ TCP receive buffer ] -> receiving application
           | used data | FREE SPACE |
                   <--- rwnd -->

```

rwnd is carried back to the sender in the TCP header.

- Receiver advertises free buffer space in the rwnd field.
- Lecture slide gives a typical default RcvBuffer example of 4096 bytes and notes many operating systems dynamically adjust buffer size.
- Sender limits unACKed/in-flight data so it does not exceed the advertised rwnd.

9.1 Flow-control variables and formulas

Receiver safety condition:

$$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$$

Advertised receive window:

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

Sender condition:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

$\text{LastByteRcvd} - \text{LastByteRead}$ is the amount currently occupying the receiver buffer. Subtracting that used space from RcvBuffer gives the free space, rwnd. The sender then keeps its unacknowledged data at or below that free-space advertisement.

9.2 The rwnd = 0 issue and one-byte probes

If the receive buffer becomes full, the receiver can advertise $\text{rwnd} = 0$. Later the application may read data and free space, but if the receiver has nothing else to send, the sender might never learn that rwnd has become nonzero. The lecture solves this with small probes.

- When rwnd is zero, the sender periodically sends a segment containing one data byte.
- The receiver ACKs the probe and includes its current rwnd.

- Eventually, when the application has freed buffer space, the ACK advertises a nonzero window and normal sending can resume.

10. TCP connection management

Before exchanging normal application data, both endpoints need to agree that a connection exists and initialize parameters such as starting sequence numbers. A handshake communicates this state explicitly.

10.1 Why a 2-way handshake is not enough

Networks can delay, duplicate, reorder, or retransmit old messages. An old connection request can therefore reappear after the original connection has finished. With only two messages, the server may believe a new connection exists even though the client is gone, creating a half-open connection or causing duplicate old data to be accepted.

10.2 TCP 3-way handshake

```

Client                                     Server
choose initial sequence x
SYN=1, Seq=x ----->
                                choose initial sequence y
                                <----- SYN=1, Seq=y, ACK=1, ACKnum=x+1
ACK=1, ACKnum=y+1 ----->

```

Now both sides know the other side is live and know each other's initial sequence numbers.

1. Client sends SYN with its initial sequence number x .
2. Server replies with SYN+ACK: its own initial sequence y and ACK $x+1$.
3. Client sends ACK $y+1$. This confirms the server's SYN and establishes the connection state.

Human analogy from the lecture: "On belay?" -> "Belay on." -> "Climbing." The third message confirms that the second side's response was actually heard.

10.3 Closing a TCP connection

TCP is full duplex, so each endpoint closes its own sending direction. A side sends $\text{FIN}=1$; the peer acknowledges that FIN. The peer can later send its own FIN, which is also acknowledged. The lecture notes that FIN and ACK can sometimes be combined and simultaneous closes can be handled.

```

Client                                     Server
FIN=1 ----->
                                <----- ACK
                                <----- FIN=1
ACK ----->

```

Each FIN closes one direction of the full-duplex connection.

11. Congestion control: what problem is TCP solving?

Congestion means too many traffic sources are sending too much data too quickly for the network to handle. The problem is inside routers/links, not merely inside one receiver.

- Manifestation 1: long queueing delays as router buffers fill.
- Manifestation 2: packet loss when router buffers overflow.

- This is different from flow control, which protects the receiver's own buffer.

11.1 Two approaches to congestion control

Approach	How congestion information reaches sender
Network-assisted	Routers give explicit feedback, possibly indicating congestion level or an allowed sending rate. Lecture examples: TCP ECN, ATM, DECbit.
End-to-end	Network does not directly report congestion. End hosts infer it from observed loss/delay. The lecture identifies this as the traditional TCP approach.

12. TCP congestion window (cwnd) and sending rate

TCP maintains a congestion window, cwnd, which limits the amount of unACKed data in flight based on the sender's estimate of network congestion. Flow control gives rwnd; congestion control gives cwnd. In practice the sender must obey the smaller of the two.

Flight size = amount sent but not yet ACKed

$\text{LastByteSent} - \text{LastByteAcked} < \min(\text{cwnd}, \text{rwnd})$

Approximate sending rate $\approx \min(\text{cwnd}, \text{rwnd}) / \text{RTT}$ bytes/s

13. Slow start

A new TCP connection does not know how much capacity is available. Slow start begins with a small cwnd and grows it exponentially while ACKs arrive successfully.

- Initial cwnd in the lecture = 1 MSS.
- cwnd roughly doubles every RTT because each arriving ACK increases cwnd, allowing more segments to be sent in the next round.
- Despite the name "slow start," the growth becomes fast because it is exponential.

RTT 0: cwnd = 1 MSS

RTT 1: cwnd = 2 MSS

RTT 2: cwnd = 4 MSS

RTT 3: cwnd = 8 MSS

... until threshold/loss changes the mode

14. ssthresh: switch from exponential to linear growth

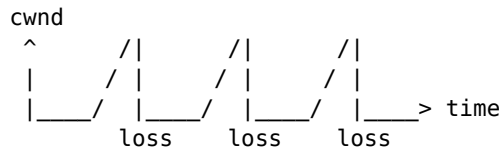
TCP uses a variable called ssthresh (slow-start threshold). After a loss event, the lecture sets ssthresh to half of the cwnd value just before the loss. On later growth, slow start is used below the threshold and congestion avoidance is used at/above it.

- On loss: $\text{ssthresh} = \text{cwnd} / 2$ (using the pre-loss cwnd).
- Below ssthresh: slow-start exponential growth.
- At/above ssthresh: congestion-avoidance linear growth.

15. Congestion avoidance and AIMD

In congestion avoidance, TCP probes for more bandwidth gradually. The lecture describes AIMD: Additive Increase, Multiplicative Decrease.

- Additive Increase: increase sending window by about 1 MSS per RTT.
- Multiplicative Decrease: when loss indicates congestion, reduce the sending window sharply.



Linear rise + sudden decrease = "sawtooth" behavior

16. TCP Tahoe vs TCP Reno loss response

Loss signal	TCP Tahoe	TCP Reno
Timeout	ssthresh = cwnd/2; cwnd -> 1 MSS; return to slow start.	ssthresh = cwnd/2; cwnd -> 1 MSS; return to slow start.
Triple duplicate ACK	Treat as serious loss: ssthresh = cwnd/2; cwnd -> 1 MSS; fast retransmit, then slow start.	Fast retransmit and fast recovery: reduce roughly to half rather than all the way to 1 MSS; then continue with linear/congestion-avoidance behavior after recovery.

- Lecture Reno diagram: on 3 duplicate ACKs, set ssthresh = cwnd/2 and cwnd = ssthresh + 3 MSS for fast recovery (noting newer Reno variants in the slide), retransmit the missing segment.
- During fast recovery, each additional duplicate ACK can increase cwnd by 1 MSS and may allow another segment to be sent.
- When a new ACK finally acknowledges the missing data, Reno leaves fast recovery and sets cwnd to ssthresh, then continues congestion avoidance.
- Any timeout drives cwnd back to 1 MSS and starts slow start again.

Lecture 12 - Formula and rule sheet

Formula / rule	Meaning / when to use
EstimatedRTT	$(1-\alpha)\text{EstimatedRTT_old} + \alpha \text{SampleRTT_current}$; typical $\alpha=0.125$.
DevRTT	$(1-\beta)\text{DevRTT_old} + \beta \text{SampleRTT_current} - \text{EstimatedRTT_old} $; typical $\beta=0.25$.
TimeoutInterval	$\text{EstimatedRTT} + 4 \text{DevRTT}$.
Timeout backoff	After timeout/retransmission, next timeout interval is doubled.
Receiver buffer safety	$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$.
Receive window	$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$.
Flow-control sender limit	$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$.
Combined flight-size limit	$\text{LastByteSent} - \text{LastByteAcked} < \min(\text{cwnd}, \text{rwnd})$.
Approx. TCP send rate	$\min(\text{cwnd}, \text{rwnd}) / \text{RTT bytes/s}$.
On loss: threshold	$\text{ssthresh} = \text{cwnd}/2$ using the cwnd just before loss.

Slow start	Start around 1 MSS; cwnd doubles approximately every RTT.
Congestion avoidance	Increase about 1 MSS per RTT.

Quick self-check

- Why is TimeoutInterval larger when RTT measurements are highly variable?
- What is the difference between rwnd and cwnd?
- Why can three duplicate ACKs trigger fast retransmit before the timer expires?
- What can go wrong with a 2-way connection handshake in a network that delays old messages?
- How do Tahoe and Reno differ after triple duplicate ACKs?

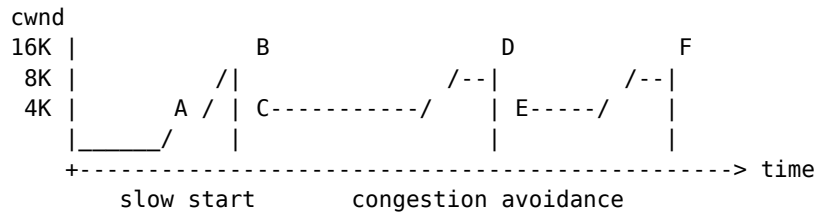
Administrative/non-study slides: The last slide is the Assignment #3 announcement; it is not additional Chapter 3 theory.

Lecture 13 - TCP Reno Problem, QUIC, Chapter Summary, and TCP Throughput

Source lecture: 1 October 2026. Only the Chapter 3 portion (through the additional transport-layer slides) is included; the later Chapter 4 network-layer slides are outside this document.

1. Worked TCP Reno window problem

The lecture begins with a graph of TCP Reno congestion window (bytes) versus time. The key skill is to identify a loss event from how sharply the window falls and how it grows afterward.



B: triple duplicate ACK -> Reno halves rather than dropping to 1 MSS

D: timeout -> cwnd drops to 1 MSS

Parts (a)-(c)

- Event B = Triple Duplicate ACK.
- Event D = Timeout.
- Between A and B, cwnd is in slow start, so it grows exponentially (roughly doubles each RTT).
Between C and D, TCP is in congestion avoidance, so cwnd grows linearly (about 1 MSS per RTT).

Part (d): time from C to D

Given: MSS = 1000 bytes; RTT = 100 ms. At C, the window has been halved from 8K to 4K bytes. At D it reaches 16K bytes. The C-to-D interval is congestion avoidance, so the window increases by 1 MSS per RTT.

1. Window increase required = $16K - 4K = 12K$ bytes.
2. One RTT increases cwnd by 1 MSS = 1000 bytes = 1K byte in the slide's arithmetic.
3. Number of RTTs = $12K / 1K = 12$ RTTs.
4. Time = $12 \times 100 \text{ ms} = 1200 \text{ ms} = 1.2 \text{ s}$.

Answer: Time from C to D = 1200 ms.

2. Why transport-layer functionality keeps evolving

TCP and UDP have been the main Internet transport protocols for decades, but no single design is ideal for every environment. The lecture lists scenarios where classic TCP/UDP behavior can be a poor fit.

Scenario	Challenge described in lecture
Long, "fat" pipes / large transfers	Many packets are in flight; a loss can shut down or sharply reduce a large pipeline.
Wireless networks	Loss can come from noisy radio links or mobility, but

	TCP may interpret loss as congestion.
Long-delay links	RTTs are extremely long, making feedback/recovery slow.
Data-center networks	Applications can be very latency sensitive.
Background traffic flows	Some flows should operate at low priority rather than competing equally.

If an application wants more than UDP but not exactly TCP's complete behavior, developers can implement transport-like functions at the application layer on top of UDP. HTTP/3 using QUIC is the lecture's main example.

3. QUIC: Quick UDP Internet Connections

QUIC is presented as an application-layer protocol that runs over UDP and supplies reliability, congestion control, security, and connection-establishment behavior needed by modern HTTP. It combines ideas studied throughout Chapter 3 rather than relying on a traditional TCP connection underneath.

- Goal in lecture: improve HTTP performance.
- Runs above UDP.
- Uses loss detection / reliable transfer with ACK-like mechanisms similar in spirit to TCP.
- Uses common congestion control, described in the slide as based on TCP NewReno.
- Establishes reliability/congestion-control state plus authentication/encryption state together in one RTT in the lecture model.
- Supports multiple application-level streams inside one QUIC connection. A stream provides reliable, in-order, bidirectional delivery for that stream.
- QUIC connection ID and per-stream IDs are carried in QUIC headers so several streams can share one connection.

3.1 Connection establishment: TCP+TLS vs QUIC

Traditional secure HTTP stack shown in lecture:

Client		Server
TCP handshake	<----->	1st serial handshake
TLS handshake	<----->	2nd serial handshake
data	<----->	

QUIC:

Client		Server
QUIC handshake	<----->	combines connection + security setup

Data can follow after the single combined handshake model

The lecture's conceptual point is latency: traditional TCP and TLS setup are shown as two serial handshakes, whereas QUIC combines the needed transport and security state establishment into one handshake.

4. Head-of-Line (HOL) blocking and QUIC streams

Head-of-line blocking means later useful data is forced to wait behind earlier missing data. With one ordered TCP byte stream, bytes after a missing segment cannot be delivered upward until the gap is repaired, even if those later bytes belong to a different HTTP request.

ONE TCP BYTE STREAM (simplified)

Request A bytes | Request B bytes | Request C bytes

X loss in A

-> TCP must repair the missing byte range before later stream bytes
can be delivered in order.**QUIC WITH SEPARATE STREAMS**

Stream A: ---X--- retransmit needed

Stream B: -----> can continue

Stream C: -----> can continue

(shared QUIC congestion control underneath, but reliability is per stream)

- In QUIC, reliable in-order delivery is handled per stream.
- A lost UDP datagram affects the QUIC streams whose data was carried in that datagram, but independent streams with available data can continue to make application-level progress.

Remember: QUIC does not mean "loss no longer matters." The key is that one stream's loss does not necessarily block delivery of already available data from unrelated streams the way one global ordered byte stream can.

5. Worked QUIC delivery-time problem

Problem from the slide: Client fetches three objects A, B, C. Each object fits in one data packet. RTT = 40 ms. QUIC needs 1 RTT for handshake before data. After that the server sends A, B, C. A has no loss; B suffers one loss; C suffers two losses. Each loss is detected after one RTT and each successful/retransmitted packet takes one RTT in the slide model.

1. Handshake cost = 1 RTT = 40 ms for every object.
2. A: no loss. Delivery = 40 ms handshake + 40 ms data = 80 ms.
3. B: one failed attempt plus one retransmission. Delivery = 40 + 40(loss interval) + 40(retransmit) = 120 ms.
4. C: two loss intervals, then a successful retransmission. Delivery = 40 + 40 + 40 + 40 = 160 ms.

Object	Losses	Delivery time
A	0	80 ms
B	1	120 ms
C	2	160 ms

Why B and C do not force A to wait: The problem is illustrating QUIC stream independence: loss associated with one object/stream does not require every other stream to wait for that stream's recovery.

6. Chapter 3 summary from the lecture

- Transport-layer principles: multiplexing/demultiplexing, reliable data transfer, flow control, congestion control.
- Internet implementations studied: UDP and TCP.

- Chapter 3 then hands off from the network edge (application + transport) toward the network core, which is why the remainder of Lecture 13 starts Chapter 4.

7. Additional Chapter 3 slides: what the FSMs are saying

The lecture includes additional reference FSMs for GBN, TCP sending, TCP handshake, and TCP closing. You do not need to memorize every line of pseudocode if you understand the state logic below.

7.1 GBN sender extended FSM

- State variables: base = oldest unACKed packet; nextseqnum = sequence number to use for the next new packet.
- New data: if $\text{nextseqnum} < \text{base} + N$, build/send it. If this is the first outstanding packet ($\text{base} == \text{nextseqnum}$), start the timer. Increment nextseqnum.
- If the window is full, refuse/delay new data until ACKs move the base.
- ACK y: set base = ACK number + 1. If no packets remain outstanding, stop timer; otherwise restart the timer for the oldest outstanding packet.
- Timeout: resend every outstanding packet from base through nextseqnum-1 and restart the timer.

7.2 GBN receiver extended FSM

- Keep expectedseqnum, the next in-order packet expected.
- If the correct expected packet arrives uncorrupted: extract/deliver it, send its ACK, increment expectedseqnum.
- Any other event: resend the ACK for the highest in-order packet; do not advance the expected sequence number.

7.3 Simplified TCP sender FSM

- NextSeqNum starts at InitialSeqNum; SendBase is the oldest unACKed byte.
- New application data: create a segment at NextSeqNum, send it, advance NextSeqNum by data length, and start a timer if none is running.
- ACK value y greater than SendBase: move SendBase to y. Stop timer if nothing remains outstanding; otherwise run the timer for the oldest outstanding segment.
- Timeout: retransmit the not-yet-ACKed segment with the smallest sequence number and restart timer.

7.4 TCP 3-way handshake state flow

CLIENT: CLOSED -> SYN_SENT -> ESTABLISHED
 SYN(x) receive SYNACK(y, x+1), send ACK(y+1)

SERVER: LISTEN -> SYN_RCVD -> ESTABLISHED
 receive SYN(x), send SYNACK(y, x+1)
 receive ACK(y+1)

7.5 TCP closing states shown in the lecture

Client side example:

ESTABLISHED -> FIN_WAIT_1 -> FIN_WAIT_2 -> TIMED_WAIT -> CLOSED

Server side example:

ESTABLISHED -> CLOSE_WAIT -> LAST_ACK -> CLOSED

TIMED_WAIT is shown as waiting for 2 x maximum segment lifetime before CLOSED.

8. Average TCP throughput from the AIMD sawtooth

Ignoring slow start and assuming the sender always has data, the lecture models cwnd as rising linearly from roughly $W/2$ to W and then being cut back after loss. The average window over that sawtooth is about $3W/4$.



Average in-flight window $\approx (W + W/2)/2 = 3W/4$

- Average TCP throughput = $(3/4) * W / \text{RTT}$ bytes/s, with W measured in bytes.

9. Worked example: TCP over a long, fat pipe

Slide example: MSS = 1500 bytes, RTT = 100 ms, desired throughput = 10 Gbps. The slide states that about 83,333 segments must be in flight and that an extremely small loss rate is required.

9.1 Required in-flight window

1. Desired rate = 10 Gbps = 10×10^9 bits/s.
2. RTT = 100 ms = 0.1 s.
3. Bits that must be kept in flight to sustain the link over one RTT = rate x RTT = $10 \times 10^9 \times 0.1 = 1 \times 10^9$ bits.
4. Convert to bytes: $1 \times 10^9 / 8 = 125,000,000$ bytes.
5. Segments in flight = $125,000,000 / 1500 = \text{about } 83,333.3$ segments, matching the slide's approximately 83,333.

9.2 Loss probability using the Mathis-style formula on the slide

TCP throughput = $(1.22 * \text{MSS}) / (\text{RTT} * \sqrt{L})$

where:

MSS = segment payload size in bytes

RTT = round-trip time in seconds

L = segment loss probability

To keep units consistent, convert the desired 10 Gbps to bytes/s: $10 \times 10^9 / 8 = 1.25 \times 10^9$ bytes/s.

1. $1.25 \times 10^9 = (1.22 \times 1500) / (0.1 \times \sqrt{L})$.
2. $1.22 \times 1500 = 1830$.
3. $0.1 \times (1.25 \times 10^9) = 1.25 \times 10^8$, so $\sqrt{L} = 1830 / 1.25 \times 10^8 = 1.464 \times 10^{-5}$.
4. $L = (1.464 \times 10^{-5})^2 = \text{about } 2.14 \times 10^{-10}$.
5. Rounded to the lecture result: L is about 2×10^{-10} - an extremely small loss probability.

Meaning: High-rate, long-RTT paths need many packets in flight. Classic loss-based TCP throughput is very sensitive to packet loss, which motivates TCP variants designed for high-speed / long-delay paths.

Lecture 13 - Formula and rule sheet

Formula / rule	Meaning / when to use
Congestion-avoidance time	If cwnd grows by 1 MSS per RTT: number of RTTs = (final cwnd - initial cwnd) / MSS; time = RTTs x RTT.
Average AIMD window	Average $\approx 3W/4$ when window oscillates between $W/2$ and W .
Average TCP throughput	Throughput $\approx (3/4) W / \text{RTT}$ bytes/s.
Bandwidth-delay amount in flight	Required bits in flight = target bit rate x RTT.
Segments in flight	$(\text{rate} \times \text{RTT} / 8) / \text{MSS}$ when rate is bits/s and MSS is bytes.
Mathis-style TCP throughput	Throughput = $1.22 \times \text{MSS} / (\text{RTT} \times \sqrt{L})$.
QUIC problem model	Total time = 1 RTT handshake + one RTT for each failed/successful data attempt in the slide's simplified example.

Quick self-check

- On a Reno cwnd graph, how can you distinguish a timeout from a triple duplicate ACK?
- Why can QUIC reduce head-of-line blocking between independent HTTP streams?
- How do you derive the number of in-flight segments needed from bandwidth, RTT, and MSS?
- Why does the Mathis formula imply that very high throughput on a long RTT path requires a very small loss probability?

Administrative/non-study slides: Lecture 13 pages after the transport-layer additional slides begin Chapter 4 (Network Layer). They are intentionally not included because this document is specifically Chapter 3 notes.

Chapter 3 - Master concept map

APPLICATIONS

|

v

TRANSPORT LAYER

|

+-- Multiplex/demultiplex -> ports and sockets

|

+-- UDP -> minimal, connectionless, checksum, no built-in reliability/CC

|

+-- Reliable transfer principles

|

 rdt1.0 -> checksum/ACK/NAK -> sequence numbers -> timer

|

 stop-and-wait -> pipelining -> GBN / SR

|

+-- TCP

 reliable ordered byte stream

 sequence + cumulative ACK + retransmission

 flow control: rwnd

 connection management: SYN/SYNACK/ACK; FIN

 congestion control: cwnd, slow start, AIMD, Tahoe/Reno

|

+-- Evolution: QUIC over UDP

 reliable streams + security + congestion control

 one combined handshake model; avoids cross-stream HOL blocking

Most exam-confusing pairs

Do not confuse	Correct distinction
IP address vs port	IP identifies host/interface; port identifies application/process endpoint on that host.
Multiplexing vs demultiplexing	Mux combines outgoing data from many sockets; demux separates incoming segments to the correct socket.
Flow control vs congestion control	Flow control protects receiver buffer; congestion control protects the network.
GBN vs SR	GBN uses cumulative progress and can retransmit a whole outstanding suffix; SR ACKs/buffers individually and retransmits only missing packets.
rwnd vs cwnd	rwnd comes from receiver free space; cwnd comes from sender's congestion-control estimate. Sender obeys the smaller one.
Timeout vs triple duplicate ACK in Reno	Timeout is more severe -> cwnd to 1 MSS; triple dup ACK -> fast retransmit/recovery and roughly halve the window.
Checksum vs reliability	Checksum only detects many bit errors; reliable delivery also needs feedback, sequence logic, retransmission, and timeout mechanisms.
TCP HOL vs QUIC streams	TCP exposes one ordered byte stream; QUIC maintains independent reliable streams so one stream's missing data need not block available data in another stream.